ANALYSIS OF SORTING ALGORITHMS

CS700 – Algorithm and Complexity Lab -3 Assignment Report

STUDENT

262IS016 - Kadam Prashant

262IS013 - Divyash Agrawal

Algorithms covered

Bubble • Insertion • Merge • Heap • Quick Sort (3 pivot strategies) • Radix

Executive Summary

This report presents a consolidated empirical study of eight sorting implementations: Bubble Sort, Insertion Sort, Merge Sort, Heap Sort, three Quick Sort pivot strategies, and Radix Sort. The structure follows the supplied reference report, while the numerical analysis in this version is based on the accompanying result files and graphs.

The benchmark data contains 8 input sizes—100, 500, 1,000, 5,000, 10,000, 20,000, 50,000 and 100,000—and three input patterns: ascending, descending and random. Each case contains six recorded runs; the supplied summary file reports the corresponding averages. This differs from the repetition count and input-size list stated in the reference report, so the present report uses the supplied result files as the authoritative source for the measured dataset.

Key Findings at n = 100,000

Algorithm	Best time (ms)	Best input	Worst time (ms)	Worst input	Random time (ms)
Bubble Sort	0.049	Ascending (sorted)	24897.298	Descending (reverse-sorted)	20353.810
Insertion Sort	0.175	Ascending (sorted)	1998.147	Descending (reverse-sorted)	1036.875
Merge Sort	7.214	Ascending (sorted)	12.990	Random	12.990
Heap Sort	4.923	Ascending (sorted)	8.153	Random	8.153
Quick Sort — First Element Pivot	5.318	Random	2455.688	Descending (reverse-sorted)	5.318
Quick Sort — Random Element Pivot	4.043	Ascending (sorted)	6.619	Random	6.619
Quick Sort — Median-of-Three Pivot	1.285	Ascending (sorted)	7.148	Descending (reverse-sorted)	5.716
Radix Sort	2.343	Random	2.373	Ascending (sorted)	2.343

At 100,000 elements, the quadratic algorithms show the expected scale-up: Bubble Sort reaches about 20.35 s on random input and 24.90 s on descending input, while Insertion Sort reaches about 1.04 s and 2.00 s respectively. In contrast, Merge Sort, Heap Sort, Random-pivot Quick Sort, Median-of-Three Quick Sort and Radix Sort remain in the millisecond range.

The supplied Quick Sort graphs show that pivot selection strongly affects robustness. First-element Quick Sort exhibits quadratic comparison growth on already ordered inputs, whereas random-pivot Quick Sort keeps the comparison count close to an n log n trajectory. Median-of-Three gives very fast ascending-input results in this dataset, but its descending-input comparison count rises substantially at the largest sizes.

Radix Sort records zero comparisons because the benchmark counts comparison operations, not digit/bucket operations. Therefore, its time-per-comparison metric is not meaningful and is treated as not applicable rather than as a performance advantage.

Contents

1. Introduction and Study Objectives
2. Experimental Setup and Data Provenance
3. Bubble Sort Analysis
4. Insertion Sort Analysis
5. Merge Sort Analysis
6. Heap Sort Analysis
7. Quick Sort — First Element Pivot Analysis
8. Quick Sort — Random Element Pivot Analysis
9. Quick Sort — Median-of-Three Pivot Analysis
10. Direct Comparison of Quick Sort Strategies
11. Radix Sort Analysis
12. Overall Algorithm Comparison
13. Execution Time vs. Comparisons
14. Cross-Algorithm Findings
15. Limitations and Interpretation Notes
16. Conclusion

1. Introduction and Study Objectives

The purpose of the study is to empirically evaluate fundamental sorting algorithms and compare measured execution time and operation counts with their expected asymptotic behavior. The supplied reference report establishes the overall experimental framing: random, sorted and reverse-sorted arrays are used to expose best-, average- and worst-case tendencies, and the execution-time curves are examined alongside comparison counts.

This expanded report adds three layers of analysis. First, it separates the behavior of each algorithm by input pattern. Second, it compares the three Quick Sort pivot policies directly. Third, it examines time per recorded comparison to show why equal comparison counts do not necessarily produce equal wall-clock time.

Algorithms Included

- Bubble Sort — quadratic comparison sort with an optimized linear best case on already sorted input.
- Insertion Sort — quadratic in the average and worst cases, with linear behavior on already sorted input.
- Merge Sort — divide-and-conquer comparison sort with O(n log n) asymptotic time across input orders.
- Heap Sort — in-place comparison sort with O(n log n) time across input orders.
- Quick Sort — evaluated with first-element, random-element and median-of-three pivot selection.
- Radix Sort — non-comparison integer sorting; for fixed-width digits, its practical growth is approximately linear in n.

The reference report describes the same algorithmic scope and the rationale for using three input patterns.

2. Experimental Setup and Data Provenance

2.1 Machine and Timing Environment

The reference report describes its measurements as averages across repeated executions and notes that system load was kept minimal.

Parameter	Value
Model	HP EliteDesk 800 G8 Tower PC
Processor	11th Gen Intel Core i5-11500 x12 (as stated in reference)
RAM	16.0 GiB
Disk	1.0 TB
Operating System	Ubuntu 24.04.3 LTS
OS Type	64-bit
Kernel	Linux 6.8.0-79-generic
Timing	C++ std::chrono high-resolution timing

2.2 Actual Supplied Benchmark Dataset

Reading the graphs: each execution-time plot should be interpreted together with its corresponding comparison-count plot. A steep time curve with a similarly steep comparison curve indicates that operation count is a major driver. A flatter comparison curve with a noticeably different time curve indicates that memory movement, copying, recursion, pivot overhead or other implementation costs contribute materially. Log–log plots help compare growth rates, but slopes estimated over a finite set of benchmark points are empirical indicators rather than mathematical proofs.

Dataset property	Observed value in supplied results
Input sizes	100, 500, 1000, 5000, 10000, 20000, 50000, 100000 elements
Input patterns	ascending, descending, random
Runs per algorithm/input-size/type	5 runs (run IDs 1–5)
Algorithms	8 implementations
Summary rows	192 averaged observations
Raw rows	1152 timing/operation observations
Time unit	microseconds in CSV; graphs label execution time in ms
Comparison count	Recorded comparison operations; Radix Sort reports 0

Data-provenance note: the reference report lists a different set of input sizes and states five repetitions. The supplied CSV files instead contain eight sizes and six runs. Because the numerical graphs are generated from the supplied result set, this report uses the CSV-derived configuration for all quantitative conclusions. The reference's machine/timing description is retained separately.

2.3 Input Construction and Interpretation

The three input categories are interpreted as follows: ascending input is already sorted, descending input is reverse sorted, and random input represents the average-case workload used in the report. The supplied result files contain these exact labels. The reference explains that these patterns are intended to expose best-case behavior for some algorithms and worst-case behavior for others.

3. Bubble Sort Analysis

Bubble Sort is a comparison-based algorithm whose dominant operation count is quadratic for unsorted inputs. The benchmark includes an optimized sorted-input path, which is visible in both the comparison and execution-time plots.

Observed behavior

- Ascending input stays close to linear growth because the algorithm can terminate after confirming that no swaps are needed.
- Descending input produces the largest comparison count and the largest execution time at high n.
- Random input is also quadratic and approaches the descending-input comparison scale as n grows.
- At n=100,000, the measured random-input time is about 20.35 s, while descending input reaches about 24.90 s.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	0.049	99,999	0.000493
Random	20353.810	4,999,863,264	0.004071
Descending (reverse-sorted)	24897.298	4,999,950,000	0.004980

Empirical Scaling Indicator

A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	0.96
Random	2.03
Descending (reverse-sorted)	1.95

Figure 1: Bubble Time

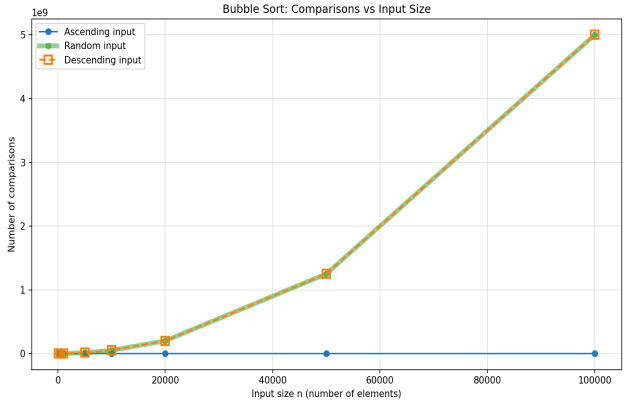
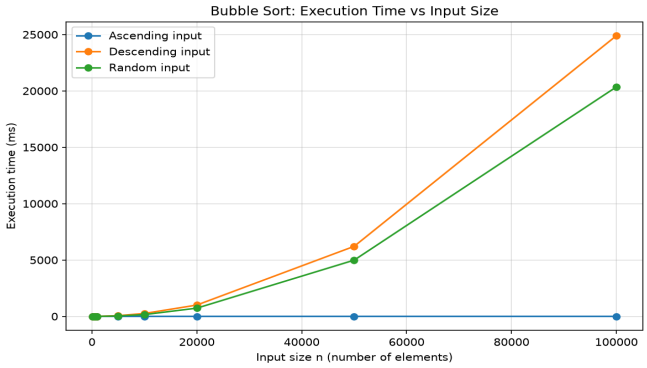


Figure 2: Bubble Comparison



Detailed interpretation: The two Bubble Sort plots should be read together. The comparison graph shows that ascending input stays almost exactly on the n scale, reaching 99,999 comparisons at n = 100,000, while descending input reaches 4,999,950,000 comparisons. Random input is also essentially quadratic, with 4,999,863,264 recorded comparisons. The fitted log–log time slopes are approximately 0.96 for ascending, 1.95 for descending and 2.03 for random input, closely matching the expected linear-versus-quadratic split over the measured range.

The timing data makes the practical consequence clearer than asymptotic notation alone. At n = 100,000, ascending input takes only about 0.049 ms, whereas descending input takes about 24,897.3 ms (24.90 s) and random input about 20,353.8 ms (20.35 s). Thus, Bubble Sort is not uniformly slow: its performance depends heavily on whether the early-termination condition can stop the passes. Once many swaps are required, the enormous number of comparisons dominates total runtime.

4. Insertion Sort Analysis

Insertion Sort performs well when elements are already close to their final positions, but repeated shifts make the worst case quadratic.

Observed behavior

- Ascending input shows linear comparison growth and very small execution time.
- Descending input produces approximately n(n−1)/2 comparisons at the largest size, creating the steepest time curve.
- Random input sits between the two extremes but remains quadratic.
- At n=100,000, random input takes about 1.04 s and descending input about 2.00 s.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	0.175	99,999	0.001753
Random	1036.875	2,498,042,892	0.000415
Descending (reverse-sorted)	1998.147	4,999,948,352	0.000400

Empirical Scaling Indicator

A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	1.01
Random	1.94
Descending (reverse-sorted)	1.97

Figure 3: Insertion Time

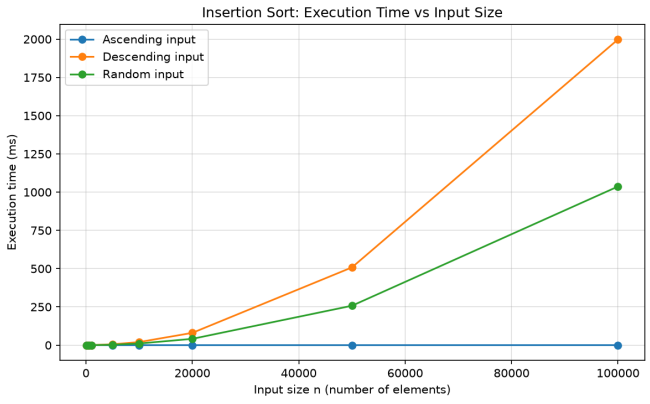


Figure 4: Insertion Comparison



Detailed interpretation: Insertion Sort exhibits the same fundamental input sensitivity as Bubble Sort, but the magnitude of the timings is substantially lower for the tested implementation. Ascending input requires only 99,999 comparisons at n = 100,000, while descending input requires 4,999,948,352 and

random input 2,498,042,892. The measured time slopes are approximately 1.01, 1.97 and 1.94 respectively, supporting linear best-case behavior and quadratic average/worst-case growth.

The difference between random and descending input is also meaningful. At the largest size, random input takes about 1,036.9 ms (1.04 s), while descending input takes about 1,998.1 ms (2.00 s). The descending case forces the maximum amount of shifting because every newly processed element must move through the already sorted prefix. This illustrates why Insertion Sort can be effective for small or nearly sorted data but becomes increasingly unsuitable as disorder and input size grow.

.

5. Merge Sort Analysis

Merge Sort divides the input recursively and merges sorted subarrays. The supplied results show much less sensitivity to input order than Bubble or Insertion Sort.

Observed behavior

- All three execution-time curves remain close, with random input generally the highest at the largest size.
- Comparison counts grow near the n log n regime rather than quadratically.
- At n=100,000, execution time ranges from about 7.21 ms (ascending) to 12.99 ms (random).
- The time-per-comparison value is relatively high because the merge process performs copying and movement work that is not counted as a comparison.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	7.214	853,904	0.008448
Random	12.990	1,536,149	0.008456
Descending (reverse-sorted)	7.481	825,290	0.009065

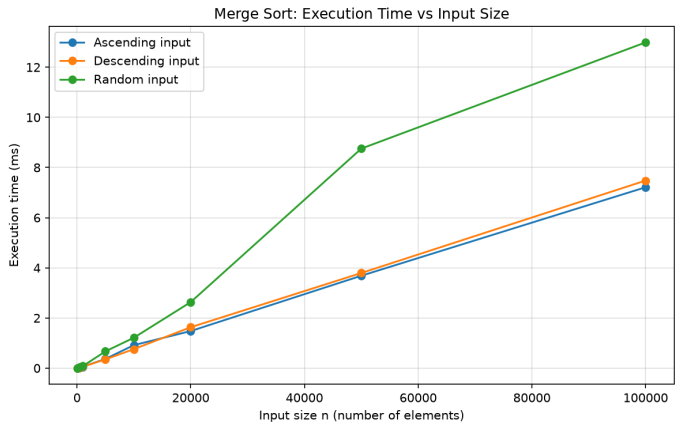
Empirical Scaling Indicator

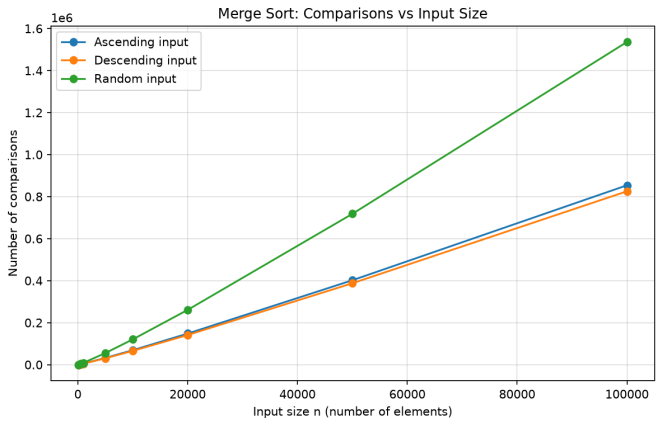
A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	1.05
Random	1.12
Descending (reverse-sorted)	1.05

Figure 5: Merge Time.

Figure 6: Merge Comparison





Detailed interpretation: Merge Sort is comparatively insensitive to input ordering. At n = 100,000, the recorded comparison counts are 853,904 for ascending input, 825,290 for descending input and 1,536,149 for random input. The differences are small relative to the billions of comparisons seen in the quadratic algorithms. The fitted time slopes are approximately 1.05, 1.05 and 1.12, consistent with n log n growth over the finite experimental range rather than a linear or quadratic regime.

The timing plot also demonstrates that comparison count alone does not determine runtime. The largest measured times are about 7.21 ms for ascending input, 7.48 ms for descending input and 12.99 ms for random input. Random input performs more comparisons and also incurs the associated merge/copy work. Merge Sort therefore offers predictable scaling, but its constant factors include temporary storage and data movement, which becomes visible in the cost-per-comparison analysis.

6. Heap Sort Analysis

Heap Sort uses a heap data structure and repeatedly extracts the maximum/minimum element. Its measured scaling is consistent across input patterns.

Observed behavior

- The three comparison-count curves nearly overlap and grow approximately as n log n.
- Execution times remain in the same millisecond-scale band across input patterns.
- At n=100,000, random input takes about 8.15 ms, while ascending and descending inputs are about 4.92 ms and 5.00 ms.
- Its relatively low time per comparison reflects in-place operation and limited data movement compared with Merge Sort.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	4.923	3,113,618	0.001581
Random	8.153	3,019,548	0.002700
Descending (reverse-sorted)	5.002	2,926,553	0.001709

Empirical Scaling Indicator

A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	1.12
Random	1.20
Descending (reverse-sorted)	1.15

Figure 7: Heap Time.

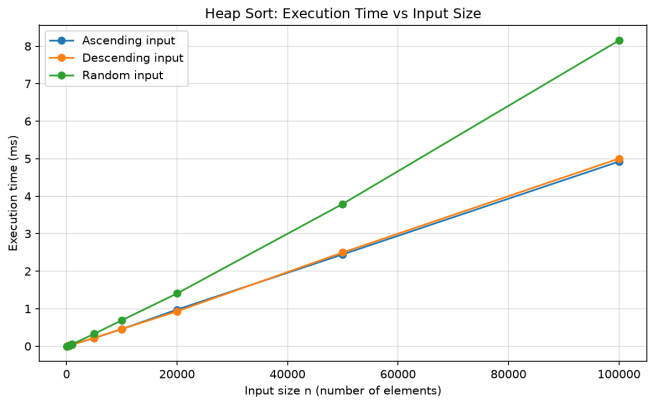
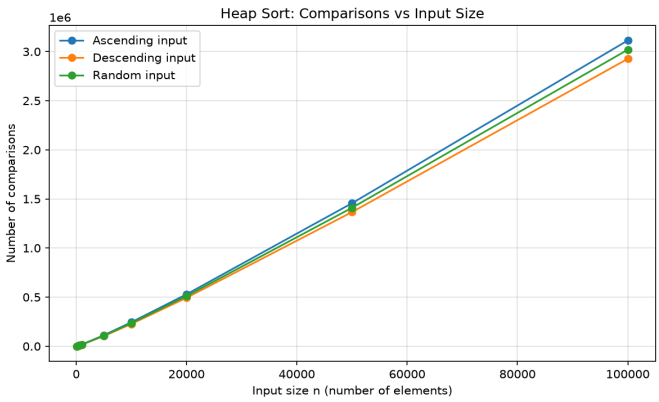


Figure 8: Heap Comparison.



Detailed interpretation: Heap Sort shows one of the most stable relationships between input order and measured work. At n = 100,000, the comparison counts are approximately 3.11 million for ascending input, 2.93 million for descending input and 3.02 million for random input. The time slopes range from about 1.12 to 1.20, characteristic of n log n growth across the tested interval. Unlike the quadratic algorithms, no input pattern causes the comparison count to explode toward n^2.

The execution-time values reinforce this stability: approximately 4.92 ms for ascending input, 5.00 ms for descending input and 8.15 ms for random input at the largest size. Random input is somewhat slower, but the difference remains small compared with the multi-second growth of Bubble, Insertion and poor-pivot Quick Sort cases. Because Heap Sort works in place, it also avoids the repeated temporary-array copying associated with Merge Sort.

7. Quick Sort — First Element Pivot Analysis

First-element Quick Sort is highly sensitive to already ordered input because the pivot can repeatedly generate a highly unbalanced partition.

Observed behavior

- Ascending and descending inputs show quadratic comparison growth.
- Random input follows a much more favourable n log n trajectory.
- At n=100,000, ascending input takes about 1.52 s and descending input about 2.46 s, versus about 5.32 ms on random input.
- The gap demonstrates that pivot choice can dominate the practical performance of Quick Sort.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	1524.163	4,999,950,000	0.000305
Random	5.318	1,987,250	0.002676
Descending (reverse-sorted)	2455.688	4,593,389,548	0.000535

Empirical Scaling Indicator

A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	1.93
Random	1.20
Descending (reverse-sorted)	1.96

Figure 9: Quick1 Time.

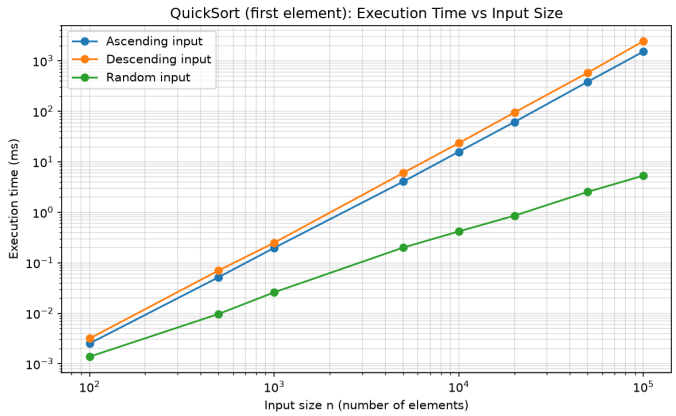
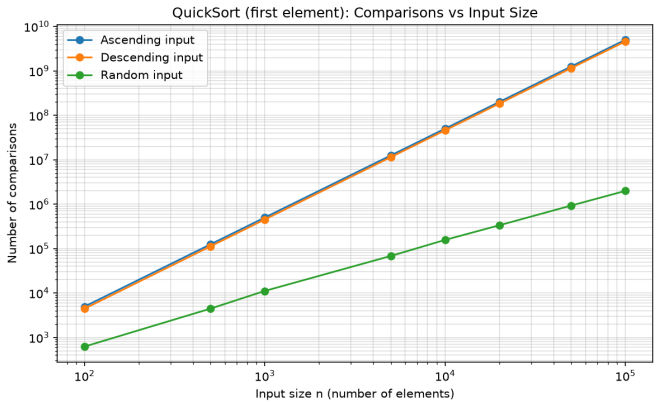


Figure 10: Quick1 Comparison



Detailed interpretation: First-element Quick Sort is the clearest demonstration that practical behavior can depend more on the input distribution than on the algorithm name alone. At n = 100,000, ascending input produces 4,999,950,000 comparisons and descending input about 4,593,389,548, both on a quadratic scale. Random input requires only about 1,987,250 comparisons. The corresponding time slopes are approximately 1.93 for ascending, 1.96 for descending and 1.20 for random input.

The wall-clock gap is dramatic. The measured times at n = 100,000 are about 1,524.2 ms for ascending input, 2,455.7 ms for descending input and only 5.32 ms for random input. The graph therefore illustrates the danger of a deterministic pivot rule that repeatedly selects an extreme element. The key lesson is not that Quick Sort itself is slow, but that pivot selection determines whether partitioning remains balanced enough to preserve n log n-like behavior.

8. Quick Sort — Random Element Pivot Analysis

Random-pivot Quick Sort reduces systematic dependence on the initial ordering by choosing the pivot randomly.

Observed behavior

- The comparison curves for all three input types are tightly grouped.
- Execution times remain in the millisecond range at n=100,000.
- At the largest size, ascending input is about 4.04 ms, descending about 4.51 ms, and random about 6.62 ms.
- The random-input time includes the overhead associated with random pivot selection.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	4.043	2,016,424	0.002005
Random	6.619	2,029,840	0.003261
Descending (reverse-sorted)	4.514	2,029,555	0.002224

Empirical Scaling Indicator

A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	1.04
Random	1.07
Descending (reverse-sorted)	0.96

Figure 11: Quick2 Time.

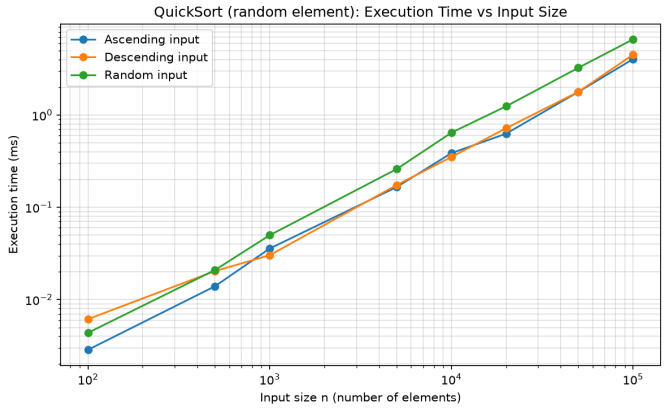
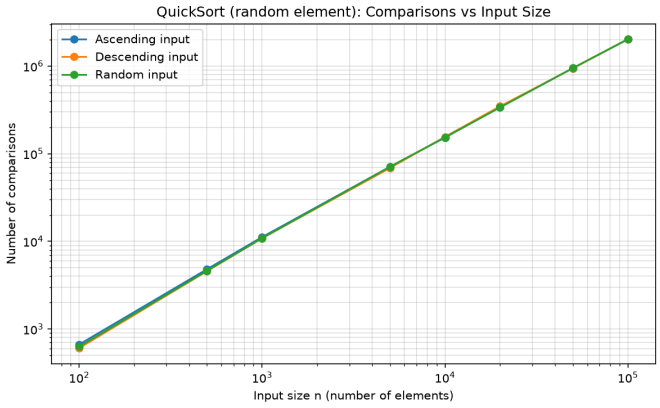


Figure 12: Quick2 Comparison.



Detailed interpretation: Random-pivot Quick Sort substantially reduces dependence on the original ordering of the data. At n = 100,000, the comparison counts are close: approximately 2.02 million for ascending input, 2.03 million for descending input and 2.03 million for random input. The time slopes are about 1.04, 0.96 and 1.07, respectively. Over this benchmark range, the three curves therefore remain close to the expected n log n regime.

The measured times at the largest size are about 4.04 ms for ascending input, 4.51 ms for descending input and 6.62 ms for random input. These values are far below the ordered-input times of the first-element strategy. The remaining differences are practical effects of partition structure, random-number generation and implementation overhead rather than evidence of a different asymptotic class.

9. Quick Sort — Median-of-Three Pivot Analysis

Median-of-Three Quick Sort chooses the median of the first, middle and last elements. It aims to avoid consistently poor pivots without requiring a random-number generator.

Observed behavior

- Ascending input is especially fast in this dataset, reaching about 1.29 ms at n=100,000.
- Random input remains near the expected n log n scaling and takes about 5.72 ms at n=100,000.
- Descending input shows a noticeable rise in comparison count and reaches about 7.15 ms at the largest size.
- The result demonstrates that deterministic pivot heuristics can greatly reduce common bad cases without eliminating every possible imbalance.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	1.285	1,601,883	0.000802
Random	5.716	1,927,806	0.002965
Descending (reverse-sorted)	7.148	12,516,251	0.000571

Empirical Scaling Indicator

A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	0.89
Random	1.14
Descending (reverse-sorted)	1.23

Figure 13: Quick3 Time.

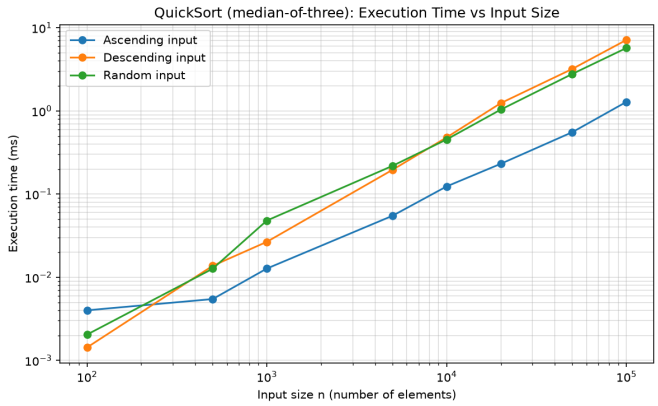
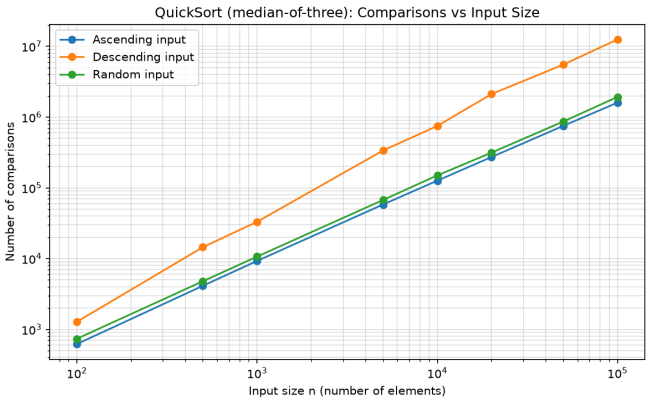


Figure 14: Quick3 Comparison.



Detailed interpretation: Median-of-Three selects a pivot using the first, middle and last elements, aiming to obtain a more representative pivot without generating a random number. In the supplied data, ascending input performs particularly well: the n = 100,000 time is about 1.29 ms with roughly 1.60 million comparisons. Random input takes about 5.72 ms with 1.93 million comparisons. The descending case is less favourable, reaching about 7.15 ms and 12.52 million comparisons.

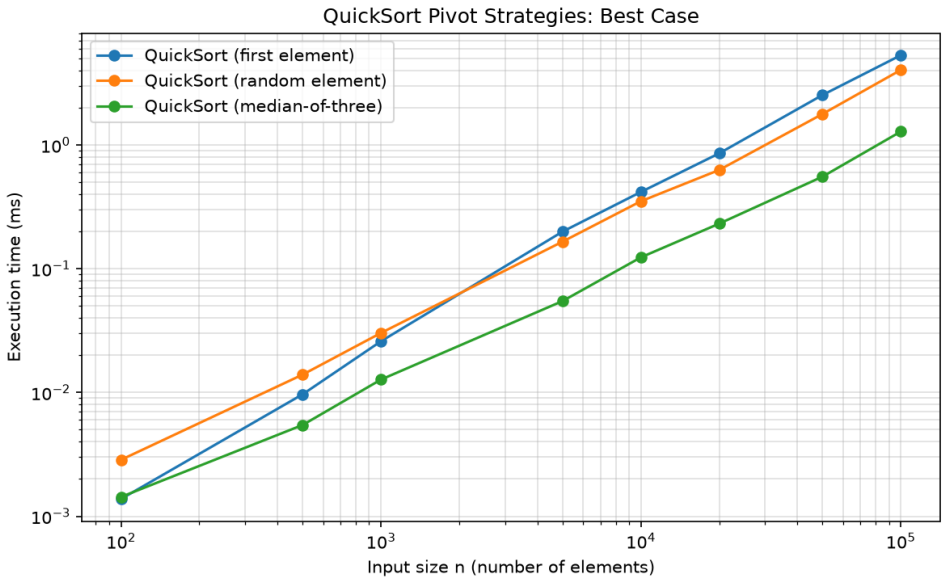
The fitted slopes make this contrast visible: ascending input is approximately 0.89, random input 1.14 and descending input 1.23 over the tested range. These finite-range slopes are empirical indicators, not proofs of asymptotic complexity. The main observation is that Median-of-Three substantially limits the extreme behavior seen with the first-element pivot, although the supplied descending results show that deterministic pivot selection can still encounter less balanced partitions.

10. Direct Comparison of Quick Sort Pivot Strategies

The three Quick Sort implementations use the same general sorting framework but differ in pivot selection. The supplied comparison graphs isolate the effect of this choice under best-case, worst-case and random-input views.

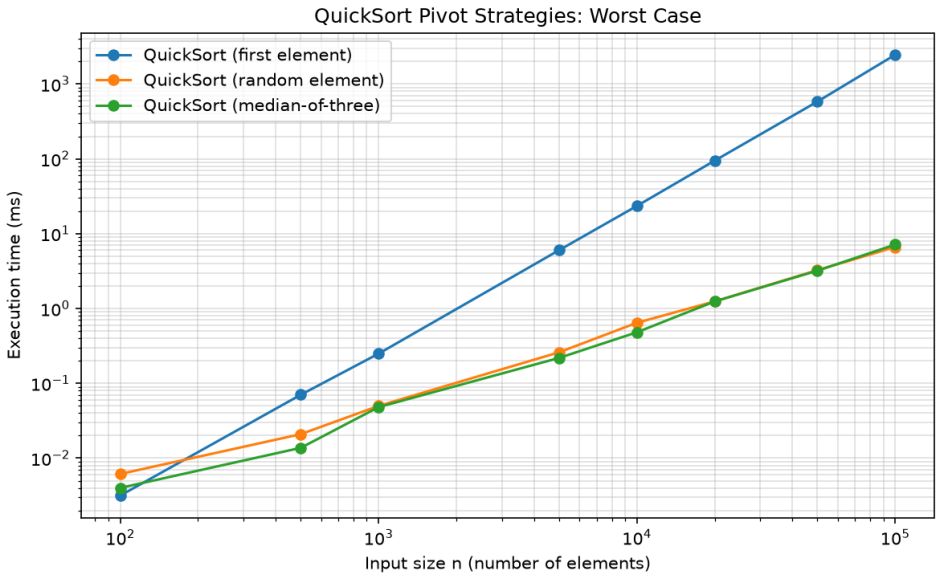
Best-case interpretation: The best-case plot isolates situations where the pivot creates a relatively balanced partition. The supplied curves show that pivot policy changes the constant factors even when the same broad n log n trend is present. Median-of-Three is particularly fast on the favourable ordered input in these measurements, while first-element and random-pivot variants remain more dependent on their specific partition sequence.

Figure 15: Best-case running time of the three Quick Sort pivot strategies versus input size.



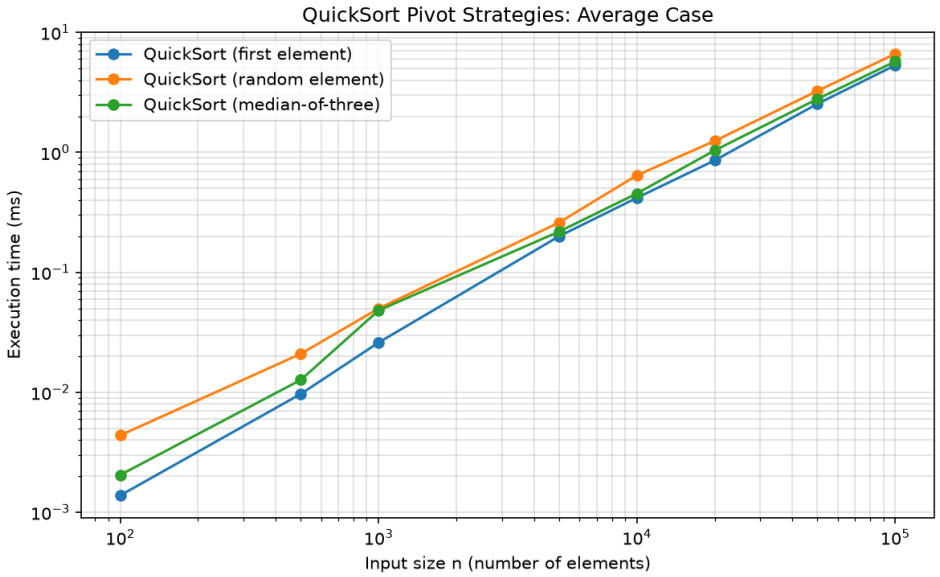
Worst-case interpretation: The worst-case plot is the most informative stress test for pivot robustness. First-element Quick Sort rises into the quadratic regime on ordered data, while the random-pivot and Median-of-Three implementations remain far below that extreme in the supplied measurements. This difference is visible in both execution time and comparison counts.

Figure 16: Worst-case running time of the three Quick Sort pivot strategies versus input size.



Average-case interpretation: On random input, all three strategies remain within a relatively narrow n log n-like band. At n = 100,000 the measured times are about 5.32 ms for first-element, 6.62 ms for random-pivot and 5.72 ms for Median-of-Three. These values show that pivot overhead and partition quality matter even when no quadratic failure occurs.

Figure 17: Average-case running time on random input versus input size.



Quick Sort comparison at n = 100,000

Pivot strategy	Ascending (ms)	Random (ms)	Descending (ms)
First Element Pivot	1524.163	5.318	2455.688
Random Element Pivot	4.043	6.619	4.514
Median-of-Three Pivot	1.285	5.716	7.148

The supplied data shows a clear separation between first-element Quick Sort and the two alternative pivot strategies on ordered inputs. First-element Quick Sort reaches more than one million

microseconds on both ordered patterns at n=100,000, while random-pivot Quick Sort remains around 4–5 ms and Median-of-Three remains below 8 ms.

The Median-of-Three implementation is particularly fast on ascending input in the supplied measurements. However, its descending-input comparison count increases sharply at the largest sizes, so the results support a nuanced conclusion: median-of-three substantially reduces the observed worst-case impact relative to the first-element strategy in this dataset, but it does not make a deterministic pivot rule immune to adverse input structure.

11. Radix Sort Analysis

Radix Sort differs fundamentally from the comparison-based algorithms because the benchmark sorts integers by processing digits. The supplied reference describes the complexity as O(d·(n+k)); when the digit count and base are fixed, this behaves approximately linearly in n.

Observed behavior

- Execution time is nearly linear with input size for all three input patterns.
- The curves almost overlap, showing very little dependence on whether the input is ascending, descending or random.
- At n=100,000, the three times are approximately 2.34–2.37 ms.
- The comparison column is zero by definition of the benchmark's counter, so comparison-based metrics do not apply to Radix Sort.

Input	Avg. time (ms)	Avg. comparisons	Time/comparison (µs)
Ascending (sorted)	2.373	0	N/A
Random	2.343	0	N/A
Descending (reverse-sorted)	2.354	0	N/A

Empirical Scaling Indicator

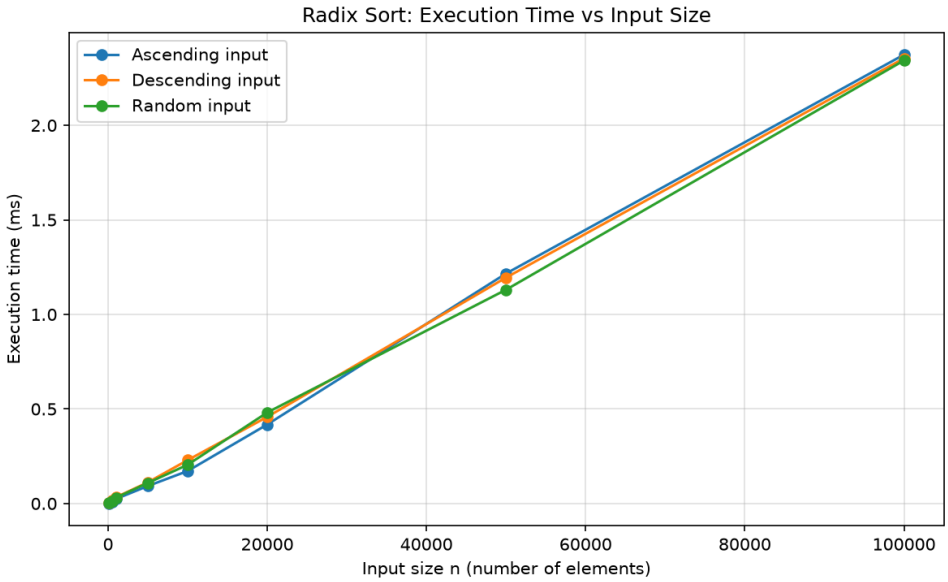
A log–log slope close to 1 suggests approximately linear scaling, while a slope near 2 suggests quadratic scaling. Values between these extremes are expected for n log n behavior over a finite benchmark range.

Input pattern	Log–log time slope
Ascending (sorted)	1.08
Random	0.98
Descending (reverse-sorted)	0.89

Detailed interpretation: Radix Sort should be interpreted separately from the comparison-based algorithms because the benchmark records zero comparison operations for every Radix Sort case. Its work is instead associated with digit processing, bucket placement and movement of integer values. Consequently, the comparison-count and time-per-comparison metrics are not applicable to this algorithm.

The timing curves are nearly independent of input order. At n = 100,000, the measured times are approximately 2.37 ms for ascending input, 2.35 ms for descending input and 2.34 ms for random input. The fitted time slopes are approximately 1.08, 0.89 and 0.98. Within experimental variation, this is close to linear growth and agrees with the supplied reference's interpretation for fixed-width integer keys.

Figure 18: Radix Time.



The supplied measurements reinforce this interpretation: at n=100,000, Radix Sort takes approximately 2.34 ms on random input, 2.37 ms on ascending input and 2.35 ms on descending input. The near-overlap of the curves indicates that element ordering has little effect on the measured runtime.

12. Overall Algorithm Comparison

The three aggregate graphs compare the algorithms by best observed case, worst observed case and random input. These plots are particularly useful because they show how asymptotic growth separates the algorithms as n increases.

Figure 19: Best-case running time of the sorting algorithms versus input size.

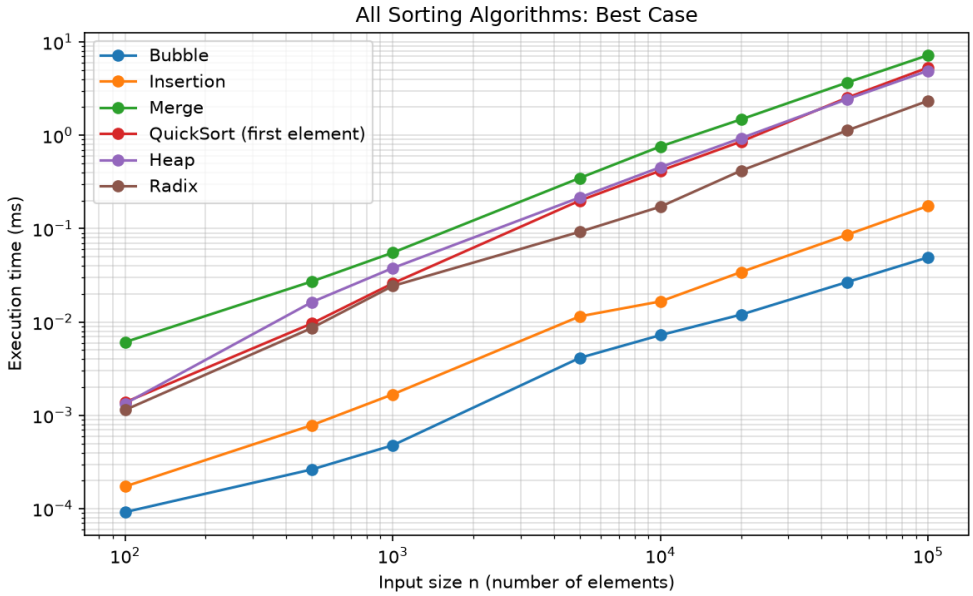
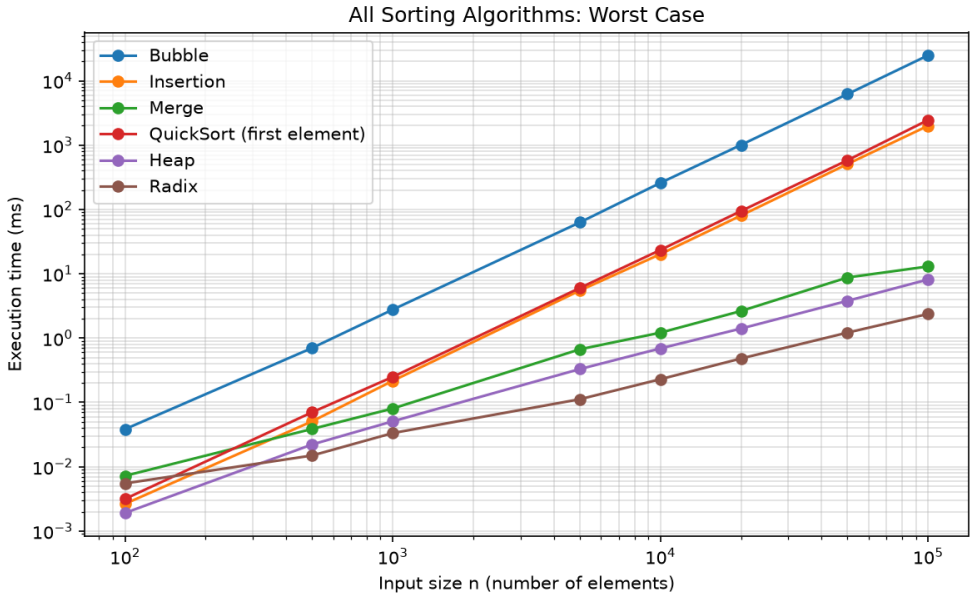


Figure 20: worst-case running time of the sorting algorithms versus input size.



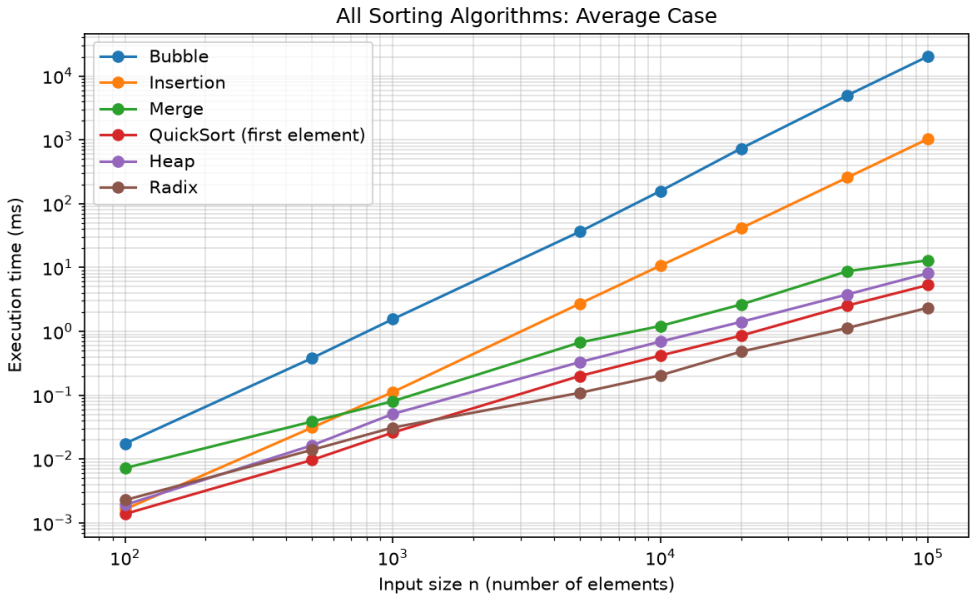
Detailed cross-algorithm interpretation: The aggregate plots make the effect of asymptotic growth visible across the full input range. At small n, several algorithms occupy the same visual neighbourhood because constant overhead dominates. As n increases, the curves separate rapidly: quadratic algorithms accelerate much faster than n log n algorithms, while Radix Sort remains close to linear for the tested integer data.

At n = 100,000 on random input, the measured times are approximately 20.35 s for Bubble Sort, 1.04 s for Insertion Sort, 12.99 ms for Merge Sort, 8.15 ms for Heap Sort, 5.32 ms for first-element Quick Sort, 6.62 ms for random-pivot Quick Sort, 5.72 ms for Median-of-Three Quick Sort and 2.34 ms for Radix Sort. These values provide a concrete scale for the theoretical complexity classes.

The best-case aggregate plot should not be used to generalize the performance of quadratic algorithms. Bubble Sort and Insertion Sort benefit strongly from already sorted input, but their growth

changes dramatically when disorder is introduced. The worst-case and random-input plots therefore provide complementary evidence and are essential for understanding scalability.

Figure 21: Average-case running time of the sorting algorithms versus input size.



12.1 Interpretation at the largest input size

At n=100,000, the benchmark shows three broad performance regimes. Bubble Sort and Insertion Sort become extremely expensive when their quadratic paths are activated. Merge Sort, Heap Sort and the two more robust Quick Sort variants stay in the millisecond range. Radix Sort also remains in the low-millisecond range and shows the least sensitivity to input ordering.

The best-case graph should be interpreted separately from general-purpose performance. Bubble and Insertion Sort are exceptionally fast on already sorted input because their measured work is approximately linear. Therefore, a fast best-case point does not imply comparable performance on random or reverse-sorted data.

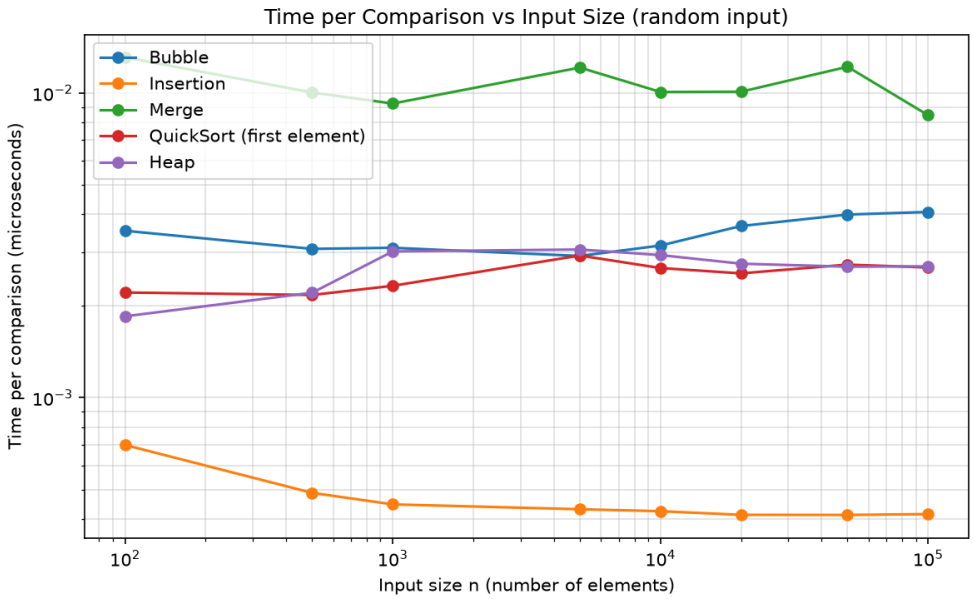
12.2 Largest-input timing snapshot

Algorithm	Ascending (ms)	Random (ms)	Descending (ms)	Random comparisons
Bubble Sort	0.049	20353.810	24897.298	4,999,863,264
Insertion Sort	0.175	1036.875	1998.147	2,498,042,892
Merge Sort	7.214	12.990	7.481	1,536,149
Heap Sort	4.923	8.153	5.002	3,019,548
First Element Pivot	1524.163	5.318	2455.688	1,987,250
Random Element Pivot	4.043	6.619	4.514	2,029,840
Median-of-Three Pivot	1.285	5.716	7.148	1,927,806
Radix Sort	2.373	2.343	2.354	0

13. Execution Time vs. Comparisons

The supplied cost-per-comparison graph asks whether the total runtime can be explained solely by the number of comparisons. It cannot. Comparison count is only one component of the work performed by a sorting implementation; memory accesses, swaps, shifts, copying, recursion and cache behavior also contribute.

Figure 22: Cost per recorded comparison for the comparison-based sorting algorithms on random input.



Detailed interpretation: The cost-per-comparison plot separates two sources of runtime: the number of recorded comparisons and the additional work performed around those comparisons. Merge Sort illustrates this clearly because merging requires copying and moving values even when the comparison counter remains relatively modest. A higher time/comparison ratio therefore reflects additional non-comparison work rather than simply more comparisons.

Quick Sort and Heap Sort appear lower in the supplied cost curve because their implementations operate largely in place. Their comparison operations are accompanied by swaps and local memory accesses rather than repeated construction and copying of temporary arrays. Bubble Sort and Insertion Sort have moderate per-comparison cost, but their enormous comparison counts in adverse cases still dominate total runtime. Radix Sort is excluded because its recorded comparison count is zero.

This analysis is important when interpreting benchmark graphs: an algorithm with fewer comparisons is not automatically faster if each comparison is surrounded by expensive memory operations, and an algorithm with a low cost per comparison can still be slow if it performs billions of comparisons. Measured runtime is therefore the combined result of operation count, memory movement, cache behavior and implementation overhead.

13.1 What the supplied data shows

- Merge Sort has the highest time-per-comparison curve in the supplied graph. Its merge phase performs substantial data movement and temporary-array copying that is not represented by the comparison counter.

- Quick Sort and Heap Sort remain comparatively low in cost per recorded comparison because their implementations operate in place and avoid the repeated copying pattern associated with Merge Sort.
- Bubble and Insertion Sort have moderate per-comparison cost, but their total comparison counts can become enormous, so their overall runtime still becomes very large.
- Radix Sort is excluded from this comparison because its comparison counter is zero; dividing its runtime by zero would not provide a meaningful metric.

The reference report makes the same qualitative distinction between comparison count and memory operations, noting that total runtime is a mixture of operation count and the efficiency of memory operations.

14. Cross-Algorithm Findings

14.1 Input sensitivity

Bubble Sort, Insertion Sort and first-element Quick Sort are highly sensitive to input ordering. For Bubble and Insertion Sort, already sorted data activates their linear best case, while reverse-sorted data produces very large quadratic work. For first-element Quick Sort, ordered data causes highly unbalanced partitions and therefore quadratic comparison growth.

14.2 Algorithms with stable scaling

Merge Sort and Heap Sort show relatively small differences between input patterns and maintain n log n-like growth. Random-pivot Quick Sort also keeps all three input patterns close together in the supplied measurements. Radix Sort shows the flattest dependence on input order and approximately linear growth over the tested range.

14.3 Wall-clock performance at n = 100,000

For random input at n=100,000, the measured order of magnitude is especially clear: Bubble Sort is around 20.35 seconds, Insertion Sort around 1.04 seconds, Merge Sort around 12.99 ms, Heap Sort around 8.15 ms, first-element Quick Sort around 5.32 ms, random-pivot Quick Sort around 6.62 ms, Median-of-Three Quick Sort around 5.72 ms, and Radix Sort around 2.34 ms.

These numbers illustrate why asymptotic growth matters: algorithms that are close at small n can become separated by orders of magnitude as the input grows.

14.4 Empirical scaling slopes

Algorithm	Ascending slope	Random slope	Descending slope
Bubble Sort	0.96	2.03	1.95
Insertion Sort	1.01	1.94	1.97
Merge Sort	1.05	1.12	1.05
Heap Sort	1.12	1.20	1.15
First Element Pivot	1.93	1.20	1.96
Random Element Pivot	1.04	1.07	0.96
Median-of-Three Pivot	0.89	1.14	1.23
Radix Sort	1.08	0.98	0.89

15. Limitations and Interpretation Notes

- The machine specifications and timing mechanism are described in the reference report, but the supplied result files do not include a machine-identifier field. The report therefore treats those specifications as the documented experimental environment rather than independently verifying them from the CSV files.
- The reference report states five repetitions and a different input-size list, whereas the supplied raw files contain six runs and eight input sizes. This report explicitly follows the supplied measurements for numerical analysis.
- The CSV files do not include the source code or compiler/build configuration. Therefore, conclusions about implementation-specific optimizations are limited to what can be inferred from the measured curves and the reference's methodological description.
- Wall-clock timings at microsecond scale can be influenced by OS scheduling, cache state, CPU frequency scaling and background activity. Repeated runs and averaging reduce—but do not eliminate—this variability.
- Radix Sort is not directly comparable through the comparison-count metric because the benchmark records zero comparisons for it. Its runtime should instead be interpreted through its digit-processing work and the measured execution time.

How to read the graphs

- Time plots use input size on the x-axis and execution time on the y-axis; several aggregate plots use logarithmic axes to make multiple orders of magnitude visible.
- Comparison plots show how the counted comparison operations scale independently of other work.
- Near-linear curves should not be interpreted as mathematically exact O(n); they indicate that the observed growth is consistent with approximately linear scaling over the measured range.
- Likewise, an observed slope near 1.1–1.2 is consistent with n log n over a finite range and should not be treated as a proof of asymptotic complexity.

16. Conclusion

The supplied benchmark results provide strong empirical evidence for the expected separation between quadratic, n log n and approximately linear sorting strategies. Bubble Sort and Insertion Sort are highly effective on their linear best cases but scale poorly once substantial rearrangement is required. Merge Sort and Heap Sort provide stable n log n behavior across input patterns. Quick Sort demonstrates that pivot selection is a decisive practical factor: first-element pivoting fails dramatically on ordered data, while random and median-of-three strategies avoid the observed quadratic blow-up in most tested cases.

Radix Sort shows nearly linear wall-clock growth and very little sensitivity to input order for the tested integer dataset. At 100,000 elements it remains around 2.3–2.4 ms across all three patterns, while the quadratic algorithms have already reached seconds.

Overall, the experiment supports a central algorithm-engineering lesson: asymptotic complexity explains large-scale growth, but real execution time also depends on constant factors and memory behavior. The comparison-count analysis demonstrates why two algorithms with similar operation counts can still have different wall-clock performance.